CSA17 – ARTIFICIAL INTELLIGENCE

ASSIGNMENT 5 – CO5

Agents, Intelligent Systems and Real-Time AI: Architecture, Engineering and Deployment

Details	Information
Course	CSA17 – Artificial Intelligence
Assignment	Assignment 5 – CO5
Part A	Design and Reasoning
Part B	Implementation, Instrumentation and Deployment Decisions
Maximum marks	50
Student name / Register number	M.Kushal /1924260243
Department / Section	AI&DS
Submission date	11/11/2026

Prepared as a structured answer guide based on the questions and evidence requirements in the supplied assignment brief.

Important note about implementation evidence

The assignment requires actual program runs, benchmark measurements, a labelled dataset split and performance evidence. This document provides reproducible designs, code templates, test plans and clearly marked example formats. Any numerical benchmark values shown as examples are illustrative only; they must not be submitted as measured results. Run the code with the selected dataset and replace every [MEASURE] field with your own output.

Contents

- A1 Agent architecture – autonomous parcel-delivery drone
- A2 Decentralised flood-rescue swarm
- A3 Generic expert-system shell
- A4 PySpark RDD data pipeline
- A5 Compressed edge-AI model
- B1 Drone and swarm simulator
- B2 Adversarial expert-system evaluation
- B3 Full model versus edge model
- B4 Chatbot and deployment decision
- Conclusion and references

PART A – TASK A1: DESIGN AN AGENT ARCHITECTURE

Autonomous parcel-delivery drone

A1.1 PEAS description

Element	Specification
Performance	Deliver the correct medical parcel; avoid collisions and restricted zones; reach destination on time; preserve battery reserve; land safely; maintain reliable communication.
Environment	City grid, buildings, roads, rooftops, landing pads, no-fly zones, moving vehicles/people, wind, weather and charging/return stations.
Actuators	Propeller/motor speed, yaw/pitch/roll control, altitude command, brake/hover, landing gear, return-to-base command, warning transmitter.
Sensors	GPS/GNSS, IMU, barometer, camera/depth sensor, lidar/ultrasonic range sensor, wind estimate, battery monitor, parcel/door status and communication receiver.

Environment classification

The environment is partially observable because sensors do not reveal every object or wind condition. It is dynamic because obstacles and weather can change during flight. It is sequential because each movement affects later route options and battery. It is stochastic because wind and sensor noise make outcomes uncertain. It is continuous in physical position and velocity, although the planning map may be discretised into grid cells. It is multi-agent when other drones or vehicles share the airspace.

Required agent types

- Model-based: remembers the map, estimated position, velocity, wind and previously observed hazards.
- Goal-based: plans a safe path from the current position to the delivery point and then to a safe landing location.
- Utility-based: compares route time, energy use and delivery progress after safety constraints are satisfied.
- Learning: can improve wind compensation or obstacle prediction from flight data, but learning must not override certified safety rules.
- Simple reflex: useful only for immediate reactions such as obstacle too close → brake/hover; it is insufficient as the complete agent.

A1.2 Percepts, state and actions

At each control cycle, the drone reads current GPS position, altitude, speed, heading, IMU acceleration, nearby obstacle distances, camera/lidar detections, wind direction and speed, battery percentage, GPS quality, no-fly-zone map and parcel status.

Its internal state stores the latest pose estimate, velocity, destination, planned route, map and no-fly polygons, obstacle tracks with timestamps, wind estimate, battery estimate, current mission mode, last safe waypoint and communication status. Old obstacle tracks expire after a defined timeout.

Actions include move to waypoint, climb, descend, turn, hover, brake, reroute, request help, return to base, select emergency landing site, land and terminate mission.

TASK A1 – DECISION RULE, PSEUDOCODE AND TRACE

A1.3 Priority and utility

Use a lexicographic priority policy: (1) prevent collision and prohibited-zone entry, (2) preserve a minimum reserve for safe landing, (3) complete delivery, (4) minimise time and energy. A lower priority objective can never compensate for violating a higher priority safety constraint.

Among routes that pass safety checks, use utility U = 0.50 × delivery progress − 0.25 × estimated energy fraction − 0.15 × travel-time fraction − 0.10 × route-risk score. The weights are design assumptions and should be tuned and validated. If battery is below the return threshold, return/land overrides the route utility.

A1.4 Hybrid sense–decide–act pseudocode

```
INITIALISE map, destination, battery_reserve, state, route
WHILE mission_status is not COMPLETE or ABORTED:
    percepts = READ_SENSORS()
    state.pose = FUSE_GPS_IMU(percepts)
    state.wind = UPDATE_WIND_ESTIMATE(percepts.wind)
    state.obstacles = UPDATE_TRACKS(state.obstacles, percepts.ranges,
                                    percepts.camera, current_time)
    state.battery = percepts.battery
    state.map = UPDATE_LOCAL_MAP(state.map, state.obstacles)

    # Reactive safety layer has highest authority
    IF DETECT_IMMEDIATE_COLLISION(state.obstacles):
        safety_action = BRAKE_AND_HOVER()
        mode = EMERGENCY_AVOID
    ELSE IF PATH_INTERSECTS_NO_FLY_ZONE(state.pose, route, map):
        safety_action = STOP_AND_REQUEST_REROUTE()
        mode = REPLAN
    ELSE IF state.battery <= RETURN_THRESHOLD:
        safety_action = RETURN_OR_SELECT_SAFE_LANDING(state.pose, map)
        mode = RETURN
    ELSE:
        safety_action = NONE

    # Deliberative layer
    IF safety_action is NONE OR mode == REPLAN:
        candidates = GENERATE_ROUTES(state.pose, destination, map)
        candidates = REMOVE_NO_FLY_AND_COLLISION_RISK_ROUTES(candidates)
        IF candidates is empty:
            planned_action = SELECT_SAFE_LANDING(state.pose, map)
        ELSE:
            route = ARGMAX_UTILITY(candidates, time, energy, risk)
            planned_action = NEXT_WAYPOINT_ACTION(route, state.pose, state.wind)

    # Arbitration: safety always wins
    IF safety_action is not NONE:
        action = safety_action
    ELSE:
        action = planned_action

    EXECUTE(action)
    LOG(percepts, state, mode, action)
    IF parcel_delivered AND safe_landing_confirmed:
        mission_status = COMPLETE
    IF emergency_landing_confirmed OR unrecoverable_fault:
        mission_status = ABORTED
END WHILE
```

A1.5 Six-step manual trace

Step	Percept	Internal state update	Acting layer / action
1	Clear path; battery 80%	At (1,1), route to (5,5)	Planning: move east
2	Wind pushes east; drift detected	Wind estimate updated; position corrected	Planning: compensate and move north-east

3	Obstacle suddenly appears 2 m ahead	Obstacle track inserted; collision risk high	Reactive: brake and hover
4	Obstacle moves across route	Track updated; old route blocked	Reactive: hover; planner computes alternate path
5	Battery falls to 24%, below 25% threshold	Mission mode becomes RETURN	Safety/battery rule: cancel delivery route and return
6	Return corridor clear; battery 20%	Safe landing pad selected	Planner guides return; land and terminate safely

The trace assumes a 25% return threshold and a reachable landing pad. In a real system, the threshold must include distance, wind, payload, battery ageing and uncertainty.

TASK A2: DECENTRALISED FLOOD SEARCH-AND-RESCUE SWARM

A2.1 Weakness of centralised control

- Single point of failure: if the control station stops, new assignments stop and drones may wait or follow stale orders.
- Communication dependence: weak or intermittent links delay updates and create inconsistent assignments.
- High communication load: a central station must receive positions and issue commands for 50 drones, increasing congestion.
- Poor fault tolerance: failure of the controller or a link can leave search cells uncovered.
- Limited scalability: central computation and message traffic grow with fleet size.

A2.2 Local frontier-and-marking strategy

Each drone maintains a local occupancy grid, a shared or periodically exchanged coverage map, and a decaying visit score for each cell. A frontier is a known free cell adjacent to an unknown cell. Each drone chooses the reachable frontier with the highest score: information gain minus travel cost, local congestion and risk. A drone broadcasts only its pose, intended target, battery state and newly observed cells when communication is available.

Per-agent pseudocode

```
EVERY control interval:
    sense nearby obstacles, water boundary, victims and neighbours
    update local occupancy grid
    mark visited cell with visit_score[cell] = current_time
    merge received neighbour map summaries
    IF battery is low:
        navigate to safe rendezvous / base
    ELSE:
        F = reachable_frontier_cells(local_map)
        FOR each cell c in F:
            score[c] = information_gain(c)
                       - alpha * travel_distance(c)
                       - beta * recent_visit_penalty(c)
                       - gamma * hazard_risk(c)
                       - delta * neighbour_target_conflict(c)
        target = highest_scoring_safe_frontier(F)
        if no safe frontier:
            explore nearest unvisited safe cell or hold position
        move one safe step toward target
    broadcast compact pose, target, battery and new map cells
```

Neighbour information

Useful neighbour messages are drone ID, position, heading, target cell, recent visited cells, battery/health and timestamp. Messages must be time-stamped; stale messages are ignored. The agent should still operate safely when no neighbour message arrives.

A2.3 Emergent behaviour

On a 5 × 5 grid, place drones D1=(1,1), D2=(5,1), D3=(3,5). Assume obstacles are absent and each drone selects the nearest safe unvisited frontier, with ties broken by lowest drone ID. D1 expands through the upper-left region, D2 through the upper-right region, and D3 through the lower-middle region. As visited cells receive penalties, drones tend to spread toward less explored areas without a central assignment.

Round	D1	D2	D3	Emergent result
0	(1,1)	(5,1)	(3,5)	Three separated starting regions
1	(2,1)	(4,1)	(3,4)	Local exploration begins
2	(2,2)	(4,2)	(2,4)	Visit penalties encourage spread
3	(1,2)	(5,2)	(4,4)	Coverage expands around each region

TASK A2 – FAILURE CASE AND TASK A3: EXPERT SYSTEMS

A2.4 Challenge: clustering and redundant coverage

Failure configuration: a narrow bridge connects two large search regions. Three drones begin on the same side and all receive the same stale map. If their target scores are equal and the tie-breaker is identical, all select the same frontier and follow one another through the bridge. This creates clustering and redundant coverage while the opposite region remains unexplored.

Predicted behaviour: repeated identical choices cause the drones to queue at the bridge. If the leading drone stops, the others may deadlock. Mitigation includes unique ID-based tie-breaking, neighbour-target penalties, reservation leases with expiry, a maximum number of drones per frontier and a timeout that triggers a different frontier.

TASK A3: Generic expert-system architecture

A3.1 Rule sets

Rules below are sample knowledge structures for demonstrating the common pattern. Real deployments require validation by pump engineers, agronomists and disaster-response specialists.

Pump-fault diagnosis rules

```
P1 IF motor_overload AND high_current THEN fault = bearing_or_impeller_jam
P2 IF low_discharge AND low_suction_pressure THEN fault = suction_blockage
P3 IF vibration_high AND bearing_temperature_high THEN fault = bearing_wear
P4 IF motor_runs AND zero_flow AND valve_closed THEN fault = discharge_valve_closed
P5 IF seal_leak AND pressure_drop THEN fault = mechanical_seal_failure
```

Crop-disease advisor rules

```
C1 IF leaf_spots AND humid_weather THEN disease = fungal_leaf_spot
C2 IF yellow_leaves AND waterlogged_soil THEN disease = root_stress
C3 IF white_powder_on_leaves THEN disease = powdery_mildew
C4 IF mosaic_pattern AND stunted_growth THEN disease = viral_mosaic
C5 IF disease = fungal_leaf_spot THEN recommend = confirm_with_lab_and_follow_local_fungicide_guidance
```

Flood-relief triage rules

```
F1 IF person_trapped AND water_rising THEN priority = CRITICAL
F2 IF medical_emergency THEN priority = CRITICAL
F3 IF elderly_or_infant AND isolated THEN priority = HIGH
F4 IF safe_shelter AND supplies_available THEN priority = LOW
F5 IF communication_lost AND last_reported_danger THEN priority = HIGH
```

TASK A3 – COMMON PATTERN AND GENERIC SHELL

Common versus domain-specific elements

Reusable across domains	Domain-specific
IF–THEN rule representation; rule parser; working memory; inference engine; conflict resolution; explanation trace; rule editor; audit log.	Vocabulary and ontology; symptoms/observations; rule conditions and conclusions; severity/priority scales; domain validation and recommended actions.

The inference engine and shell can be reused without changing engine code when the rule language and data schema remain compatible. Only the knowledge base and domain vocabulary need to change. Domain experts must approve rules before operational use.

A3.2 Components and exchanged data

Component	Role / exchanged information
Knowledge base	Stores rules, priorities, confidence, provenance and version. Supplies matching rules to inference engine.
Working memory	Stores current case facts, derived facts, timestamps and confidence. Receives sensor/user facts and inferred conclusions.
Inference engine	Matches rule conditions against working memory; selects and fires rules; records conflict-resolution decisions.
Explanation facility	Uses fired-rule trace to answer HOW a conclusion was reached and WHY evidence was requested.
Knowledge-acquisition interface	Allows experts to add, edit, validate, version and approve rules without editing inference-engine code.

Inference cycle pseudocode

```
INPUT case_facts
working_memory = NORMALISE(case_facts)
fired_trace = []
REPEAT:
    agenda = []
    FOR each rule in knowledge_base:
        IF rule conditions match working_memory:
            IF rule conclusion is not already supported:
                add rule to agenda
    IF agenda is empty: BREAK
    selected = RESOLVE_CONFLICT(agenda,
                priority_then_specificity_then_recency)
    new_fact = FIRE(selected)
    ADD new_fact to working_memory
    APPEND (selected, matched_facts, new_fact) to fired_trace
UNTIL no new facts or maximum inference steps reached
RETURN conclusions, fired_trace
```

Conflict resolution

A practical deterministic policy is: highest expert-assigned priority first; if tied, choose the rule with more specific conditions; if still tied, choose the rule supported by the most recent evidence. The system must preserve competing conclusions when evidence conflicts rather than silently hiding disagreement.

A3.3 HOW, WHY and knowledge acquisition

- HOW explains the chain of rules and facts that produced a diagnosis, allowing experts to audit the result.
- WHY explains why a question or measurement is being requested, helping the user provide relevant evidence.
- Knowledge acquisition lets domain experts update knowledge as equipment, crops or emergency procedures change.
- Versioning, approval and regression tests prevent a new rule from unexpectedly damaging earlier diagnoses.

TASK A4: DISTRIBUTED DATA PIPELINE WITH PYSPARK RDDs

Problem and data model

Input contains approximately 10 million daily records: (sensorId, timestamp, value). Required outputs are per-sensor mean, per-sensor maximum and the ten noisiest sensors. For this design, noise is defined explicitly as the standard deviation of readings per sensor. If the instructor intends another definition, such as rapid fluctuation count, replace the noise statistic and document it.

RDD pipeline pseudocode

```
raw = sc.textFile(input_path)
parsed = raw.map(parse_record)                  # narrow transformation
valid = parsed.filter(is_valid)                 # narrow transformation
keyed = valid.map(lambda r: (r.sensorId,
                        (r.value, r.value, r.value, 1)))
# tuple: (sum, max, sum_of_squares, count)
stats = keyed.reduceByKey(combine_stats)        # wide: shuffle by sensorId
stats.cache()                                   # reused by mean/max/noise
summary = stats.mapValues(compute_mean_max_std)  # narrow transformation
summary.saveAsTextFile(output_path)             # action; evaluation occurs
top10 = summary.takeOrdered(10, key=lambda x: -x[1].stddev)  # action
print(top10)
```

Transformations, actions and evaluation

Operation	Type	Evaluation / purpose
textFile	RDD creation	Creates lineage; no full computation yet.
map / filter / mapValues	Transformation; narrow	Builds dependency graph; normally no shuffle.
reduceByKey	Transformation; wide	Groups values by key and shuffles data across partitions.
cache / persist	Persistence request	Materialisation occurs when an action computes the RDD.
saveAsTextFile / takeOrdered	Action	Triggers execution of the required lineage.

RDD transformations are lazy. Spark builds a lineage graph first; an action triggers jobs and stages. A wide transformation such as reduceByKey creates a shuffle boundary. Narrow transformations can be computed from a small number of parent partitions without redistributing all records.

Caching and recovery

Cache the aggregated per-sensor statistics only if multiple downstream actions reuse them. Caching the raw 10-million-row input may consume substantial memory and is wasteful if it is read once. If the raw data is reused by several analyses, persist it only after filtering and parsing, subject to memory measurements.

RDD lineage records how each partition was derived. If a partition is lost, Spark can recompute it from parent partitions and transformations. Reliable input storage and deterministic parsing are still required; lineage is not a substitute for backups.

TASK A5: COMPRESSED EDGE-AI MODEL

Selected use case: real-time emotion detection

The selected case is facial-expression category detection on an edge camera. The model predicts a limited set of visible expression labels from a face crop. It must not claim to know a person's internal emotional state with certainty. Results are probabilistic and may be affected by lighting, pose, culture, occlusion and dataset bias.

A5.1 Full-model resource estimate

Illustrative full-model choice: a convolutional neural network with approximately 25 million parameters and 224 × 224 RGB input. With FP32 weights, parameter storage is approximately 25,000,000 × 4 bytes = 100 MB, excluding activations, runtime and temporary buffers. A model of this scale may exceed a small edge device's memory and 50 ms latency target, particularly when image preprocessing and camera I/O are included.

Resource	Estimate / interpretation
Parameters	25 million (assumed architecture estimate)
Weight memory	About 100 MB in FP32; actual file size depends on layers and serialization
Compute	Must be measured for the selected architecture; report MACs/FLOPs from model summary
Latency	Measure end-to-end on target device; do not infer from server timing
Peak memory	Measure weights + activations + runtime buffers

A5.2 Compression design

- Use a mobile-oriented depthwise-separable CNN rather than a large conventional CNN.
- Apply structured channel pruning to remove filters with low contribution.
- Quantise weights and activations from FP32 to INT8 where the device supports efficient integer kernels.
- Use a small face crop and resize once; avoid repeated image conversion.
- Use an early-exit confidence threshold: if the compact model is confident, return; otherwise run a larger fallback only if the latency budget permits.

Compressed inference pseudocode

```
frame = CAMERA_READ()
face = DETECT_AND_CROP_FACE(frame)
if face is None:
    return "no face / request another frame"
x = RESIZE_AND_NORMALISE(face)
x_int8 = QUANTISE_INPUT(x)
scores = INT8_EDGE_MODEL(x_int8)
if MAX(scores) >= CONFIDENCE_THRESHOLD:
    result = ARGMAX(scores)
else:
    result = "uncertain"
return result, MAX(scores)
```

The compact model saves weight memory, memory bandwidth and multiply-accumulate cost. A separate face detector can still dominate total latency, so measure the entire pipeline.

TASK A5 – TRADE-OFFS AND PART B IMPLEMENTATION PLAN

A5.3 What may be sacrificed?

- Accuracy: pruning and quantisation can reduce subtle feature representation, especially for minority classes.
- Robustness: performance may fall under low light, camera blur, side profiles, masks and unfamiliar faces.
- Calibration: quantisation may change confidence scores, making the old confidence threshold unreliable.
- Explainability: a smaller neural model is not automatically interpretable; provide class confidence and, where appropriate, a validated visual explanation.

Detect losses using a held-out, subject-independent test set; per-class precision/recall; confusion matrix; calibration error; low-light and pose stress tests; and latency/memory measurements on the target device. Compare the compressed model to the full model on exactly the same examples.

Part B – experimental protocol

The assignment requires source code, test input, expected behaviour, actual output, performance metrics and interpretation. Keep a fixed random seed, log software/hardware versions, preserve the test set, and report both average and tail latency. Do not write a benchmark number until the program has actually been run.

Evidence item	What to record
Reproducibility	Python, Spark and library versions; CPU/GPU/NPU; RAM; operating system; seed.
Correctness	Dataset version; train/validation/test split; class distribution; metrics and confusion matrix.
Efficiency	Model size; peak memory; mean and p95 latency; throughput; energy if measurable.
Stress testing	Boundary cases, corrupted input, missing values, sensor noise and failure scenarios.
Interpretation	Which requirement is met, what fails, likely cause and next engineering change.

Metric definitions

Accuracy = (TP + TN) / all cases. Precision = TP / (TP + FP). Recall = TP / (TP + FN). F1 = 2 × precision × recall / (precision + recall). For imbalanced classes, report macro-F1 and per-class values. ROC-AUC or PR-AUC should be computed from continuous scores, not hard labels.

PART B – TASK B1: DRONE AGENT GRID SIMULATOR

B1 implementation structure

Separate the simulator from the agent. The simulator owns the grid, obstacles, battery drain and movement validation. The agent receives percepts and returns an action. This separation makes it possible to test the same agent in different worlds.

```
class DroneAgent:
    def __init__(self, start, goal, battery=100):
        self.pos = start
        self.goal = goal
        self.battery = battery
        self.visited = {start}
        self.mode = "DELIVER"

    def decide(self, percept):
        if percept["obstacle_ahead"]:
            return "HOVER"
        if self.battery <= percept["return_threshold"]:
            self.mode = "RETURN"
        if self.mode == "RETURN":
            return percept["safe_return_action"]
        return percept["next_safe_route_action"]

class GridWorld:
    def step(self, action):
        # Validate boundary, obstacle and no-fly constraints.
        # Update position, battery, time and event log.
        return self.make_percept()
```

Test scenarios

Scenario	Input setup	Expected behaviour
Normal flight	Clear grid; sufficient battery; reachable goal	Drone follows route, delivers parcel and lands.
Sudden obstacle	Insert obstacle in next cell after movement begins	Reactive layer hovers/brakes and planner selects a safe detour.
Boundary: low battery	Battery at or below return threshold	Delivery is abandoned; drone returns or selects safe landing.
Goal blocked	Goal cell becomes unavailable	Agent replans, requests assistance or safely aborts; it must not enter blocked cell.

Behaviour log template

Time	Position	Battery	Percept/event	Mode	Action	Result
0	(1,1)	100	Clear	DELIVER	MOVE_EAST	Moved
1	[MEASURE]	[MEASURE]	[EVENT]	[MODE]	[ACTION]	[OBSERVED RESULT]
2	[MEASURE]	[MEASURE]	[EVENT]	[MODE]	[ACTION]	[OBSERVED RESULT]

The first row is an illustrative log format, not a claim about an executed simulator. Fill later rows from the program's real output.

TASK B1 – TRACE COMPARISON AND SWARM IMPROVEMENT

B1.2 Compare hand trace with program trace

Align the manual six-step trace from A1.5 with simulator timestamps. Compare sensor values, state updates, chosen layer and action. Differences can arise from grid discretisation, tie-breaking, obstacle-detection radius, battery drain per move or asynchronous events. Explain each difference rather than changing the manual trace after seeing the output.

Comparison point	Manual design	Program run / explanation
Obstacle detection	At 2 m ahead	[ACTUAL DETECTION RANGE]; explain discretisation
Battery threshold	25% assumed	[ACTUAL THRESHOLD]; confirm same setting
Avoidance action	Brake and hover	[ACTUAL ACTION]; note action arbitration
Reroute	Alternate route	[ACTUAL ROUTE]; explain tie-breaking

B1.3 Multi-drone coordination

Each drone independently selects a frontier using information gain, travel cost, hazard risk and neighbour-target conflict. A lightweight reservation table prevents multiple drones from selecting the same target simultaneously. Reservations expire if the drone fails or communication is lost.

```
FOR each drone d:
    percept = sense_local_world()
    update_local_map(percept)
    receive_neighbour_summaries_if_available()
    if battery_low(d):
        target = nearest_safe_return_point(d)
    else:
        candidates = reachable_frontiers(d.local_map)
        target = argmax(information_gain
                        - distance_cost
                        - hazard_penalty
                        - reserved_target_penalty)
    reserve(target, expiry_time)
    execute_one_safe_step_toward(target)
    broadcast_compact_status()
```

B1.4 Experimental design

Run at least 30 independent random seeds. Test N = 5, 10, 25 and 50 drones. For each N, compare single-drone baseline with swarm coordination where applicable. Inject failures at 0%, 10% and 30% of drones. Keep map size, obstacle distribution, battery model and victim distribution consistent across paired runs.

Metric	Definition
Mission success	Completed assigned rescue/delivery missions ÷ total missions
Coverage	Unique safe cells visited ÷ total reachable safe cells × 100
Redundancy ratio	Total visits beyond first visit ÷ total visits
Battery used	Initial battery minus final battery, averaged per active drone
Decision latency	Time to compute one action; report mean and p95
Robustness	Change in success and coverage under injected drone failures

TASK B1 – RESULTS REPORTING AND SCALING

Required results table

N	Failure %	Success %	Coverage vs time	Redundancy	Battery used	Mean / p95 latency
5	0	[MEASURE]	[MEASURE]	[MEASURE]	[MEASURE]	[MEASURE]
5	10	[MEASURE]	[MEASURE]	[MEASURE]	[MEASURE]	[MEASURE]
5	30	[MEASURE]	[MEASURE]	[MEASURE]	[MEASURE]	[MEASURE]
10	0/10/30	[MEASURE]	[MEASURE]	[MEASURE]	[MEASURE]	[MEASURE]
25	0/10/30	[MEASURE]	[MEASURE]	[MEASURE]	[MEASURE]	[MEASURE]
50	0/10/30	[MEASURE]	[MEASURE]	[MEASURE]	[MEASURE]	[MEASURE]

Interpretation guide

- Coverage may rise faster with more drones initially, but gains can flatten when drones compete for the same narrow routes.
- Redundancy can increase if map sharing is delayed or target reservations are ineffective.
- Decision latency may rise with larger neighbour lists and map merging; local summaries and bounded message size reduce this cost.
- A swarm may degrade more gracefully than a central controller under drone failure, provided local autonomy and safe return are implemented.
- Report mean and standard deviation across 30 seeds; include confidence intervals where possible.

Plot recommendations

Create one line graph of coverage percentage versus simulation time for each fleet size. Create a grouped bar chart of mission success under 0%, 10% and 30% failures. Create a line or box plot for decision latency as N increases. Use the same axes and definitions for all compared systems.

B1 conclusion template

In our experiment, the swarm [improved/did not improve] coverage compared with the baseline. At N = [MEASURE], the success rate was [MEASURE] and p95 decision latency was [MEASURE]. The main limitation was [MEASURED CAUSE]. The next improvement should target [SPECIFIC BOTTLENECK].

PART B – TASK B2: PUMP-FAULT EXPERT SYSTEM

B2.1 Rule base with five fault types

Use at least 15 rules and five fault types. The following sample rule base is a starting structure. Adapt feature names to the supplied or instructor-labelled cases and have an expert validate the conditions.

```
R01 IF high_vibration AND hot_bearing THEN bearing_wear
R02 IF noisy_bearing AND rising_temperature THEN bearing_wear
R03 IF shaft_play AND vibration_high THEN bearing_wear
R04 IF low_suction_pressure AND low_flow THEN suction_blockage
R05 IF inlet_filter_alarm AND low_suction_pressure THEN suction_blockage
R06 IF suction_vacuum_high AND flow_low THEN suction_blockage
R07 IF motor_current_high AND motor_overload THEN impeller_jam
R08 IF motor_trip AND current_spike THEN impeller_jam
R09 IF abnormal_motor_sound AND current_high THEN impeller_jam
R10 IF seal_leak AND pressure_drop THEN seal_failure
R11 IF visible_seal_damage AND leakage THEN seal_failure
R12 IF leakage_rate_high AND seal_temperature_high THEN seal_failure
R13 IF valve_closed AND motor_running AND zero_flow THEN valve_closed_fault
R14 IF discharge_pressure_high AND flow_zero THEN valve_closed_fault
R15 IF actuator_position_closed AND no_discharge THEN valve_closed_fault
```

Shell behaviour

Represent each rule as a record containing rule_id, conditions, conclusion, priority, confidence, active flag and version. The inference engine must not contain pump-specific if/else statements. It loads the rule records and matches them against facts.

Training and hold-out protocol

The assignment specifies 30 expert-labelled cases: use only 15 to construct or tune the rule base and keep the other 15 untouched for hold-out evaluation. Record the exact case IDs in each split. Avoid changing rules after inspecting hold-out errors; if rules are revised, create a new validation set or clearly report the iterative process.

Evaluation item	Report
Training cases	15 IDs and class distribution
Hold-out cases	15 IDs and class distribution
Accuracy	[MEASURE]
Per-fault precision / recall	[MEASURE FOR EACH OF FIVE FAULTS]
Rules fired per case	Mean, median and range
Explanations	At least one HOW and one WHY example

Metric caution

With only 15 hold-out cases, each single case changes overall accuracy by about 6.67 percentage points. Report the confusion matrix and raw counts alongside percentages. A small hold-out set gives a noisy estimate and should not be presented as proof of field reliability.

TASK B2 – ADVERSARIAL INPUTS AND CONFLICT RESOLUTION

B2.2 Adversarial case design

Case	Input facts	Predicted weakness	What to compare
Conflicting rules	high_vibration, hot_bearing, motor_current_high, motor_overload	Bearing-wear and impeller-jam rules both fire	Priority vs specificity vs recency
Missing evidence	low_flow only; no suction or valve evidence	Insufficient facts may cause unsupported diagnosis	Abstain/ask-more vs forced best match
Rule-order dependence	Facts satisfy two equal-priority rules	First-match engine may return different output when rules are reordered	First-match vs deterministic conflict strategy
Noisy evidence	temperature sensor stuck high	False evidence activates bearing-wear rules	Confidence and sensor-quality gating

Before execution, write the expected result for each case. Then run each strategy and record actual conclusion, fired rules and explanation. A safe expert system should be able to return 'insufficient evidence' or multiple candidate faults.

Conflict-resolution strategies

- Specificity: prefer the rule with the greatest number of matched conditions.
- Priority: prefer a rule assigned higher expert-approved importance.
- Recency: prefer rules supported by the newest relevant observations.

Recommended combined policy: priority first, then specificity, then recency, with rule ID as a final deterministic tie-breaker. This avoids accidental dependence on the order in which rules happen to be stored.

HOW and WHY explanation examples

HOW example: 'Bearing wear was suggested because R01 matched high vibration and hot bearing. The system also checked R07, but motor overload was absent.'

WHY example: 'Please provide motor current because a high-current reading helps distinguish an impeller jam from a suction-side blockage.'

B2.3 Runtime knowledge acquisition

```
def add_or_edit_rule(rule_store, rule_id, conditions, conclusion, priority):
    validate_rule_syntax(conditions, conclusion)
    rule_store[rule_id] = {
        "conditions": conditions,
        "conclusion": conclusion,
        "priority": priority,
        "version": next_version(rule_id)
    }
    save_rule_store(rule_store)  # no inference-engine code change

# Regression procedure:
old_results = diagnose_all(regression_cases, old_rule_store)
update_rule_store()
new_results = diagnose_all(regression_cases, updated_rule_store)
report_changed_cases(old_results, new_results)
```

Regression report must list case ID, old diagnosis, new diagnosis, rules fired and whether the change was intended. Keep an audit trail and require expert approval before deploying a changed rule.

PART B – TASK B3: FULL MODEL VERSUS EDGE MODEL

B3 applied case and dataset

Selected case: real-time emotion-expression classification, matching Task A5. Choose a public, properly licensed dataset approved by the instructor. One possible research dataset is FER-2013; verify its current download terms, class labels and usage restrictions before use. Record the exact source URL, version, retrieval date and any preprocessing.

A leakage-safe split should be subject-independent where subject identifiers exist, or source-wise when images come from known sources. If identities or source groups are not available, state this limitation and use a fixed stratified split, ensuring duplicates and near-duplicates do not cross partitions.

B3.1 Training and evaluation workflow

1. Inspect labels, class counts, image dimensions and missing/corrupt files.
2. Create train, validation and test splits before augmentation.
3. Train the full model on training data; select hyperparameters using validation data only.
4. Freeze the selected model and evaluate once on the untouched test set.
5. Save predictions, class probabilities, confusion matrix and per-class metrics.
6. Repeat evaluation for the compressed model using the same test examples.

Suggested model comparison

Model	Purpose	Measure
Full CNN	Reference/server model	Accuracy, macro-F1, PR/ROC-AUC, size, latency
Pruned CNN	Remove redundant channels	Same metrics and resource measurements
INT8 mobile CNN	Edge deployment candidate	Same metrics plus p95 latency and peak RAM

Confusion matrix interpretation

Rows should represent true labels and columns predicted labels. Large off-diagonal values identify classes the model confuses. Report macro-F1 because it gives each class equal weight. For multi-class emotion labels, compute one-vs-rest ROC-AUC or PR-AUC per class and state the averaging method.

Metric report template

Metric	Full model	Edge model
Accuracy	[MEASURE]	[MEASURE]
Macro precision	[MEASURE]	[MEASURE]
Macro recall	[MEASURE]	[MEASURE]
Macro F1	[MEASURE]	[MEASURE]
ROC-AUC / PR-AUC	[MEASURE]	[MEASURE]
Model size (MB)	[MEASURE]	[MEASURE]
Mean / p95 latency (ms)	[MEASURE]	[MEASURE]

TASK B3 – COUNTEREXAMPLES AND DATA SCALING

B3.2 Find and categorise five mis-predictions

Select at least five incorrect predictions from the test set. Do not invent examples: save the image ID, true label, predicted label, confidence and a short visual/data-quality description. Do not expose identifiable faces in a public report without appropriate permission.

Error ID	Possible category to investigate	Evidence to record
E1	Low illumination	Image ID, brightness measure, true/predicted class
E2	Partial occlusion	Occluded region and confidence
E3	Side pose / unusual angle	Pose estimate and class confusion
E4	Motion blur / low resolution	Blur or sharpness measure
E5	Ambiguous expression / label noise	Annotator disagreement or uncertain label

These are categories to search for, not claims that the model necessarily made these errors. If the five observed errors belong to other categories, report the actual categories.

B3.3 PySpark scale experiment

Use the RDD pipeline from A4 for feature extraction or batch scoring. Generate or select 10^5, 10^6 and 10^7 records. Keep the data content and operation identical while varying partition count across 1, 2, 4 and 8.

```
for size in [10**5, 10**6, 10**7]:
    for partitions in [1, 2, 4, 8]:
        rdd = load_records(size).repartition(partitions)
        start = monotonic_time()
        result = rdd.map(parse_and_extract).reduceByKey(aggregate)
        result.count()  # action triggers execution
        elapsed = monotonic_time() - start
        log(size, partitions, elapsed)
```

Scaling results template

Records	Partitions	No cache (s)	Cached (s)	Shuffle count	Notes
100,000	1	[MEASURE]	[MEASURE]	[MEASURE]	[NOTES]
100,000	2	[MEASURE]	[MEASURE]	[MEASURE]	[NOTES]
1,000,000	4	[MEASURE]	[MEASURE]	[MEASURE]	[NOTES]
10,000,000	8	[MEASURE]	[MEASURE]	[MEASURE]	[NOTES]

More partitions may not improve speed when tasks become too small, scheduling overhead dominates, the cluster has too few cores, input data is skewed, or shuffle/network costs dominate. Caching helps only when the same computed RDD is reused enough to repay the memory and cache-write cost.

TASK B3 – COMPRESSED MODEL AND EDGE BENCHMARK

B3.4 Compression experiment

Train or convert the compact model using representative calibration data. For INT8 quantisation, use training or calibration data only; never use test labels to tune quantisation. Compare full and compressed models on the same held-out test set.

Latency and throughput

Measure end-to-end latency after warm-up. Record per-sample times and report mean and 95th percentile. Throughput = number of processed samples divided by elapsed seconds. Include preprocessing, face detection, inference and postprocessing if those operations are part of the deployed system.

Measure	Full model	Compressed model	How to obtain
Mean latency (ms)	[MEASURE]	[MEASURE]	Average of repeated per-sample timings
P95 latency (ms)	[MEASURE]	[MEASURE]	95th percentile of latency list
Throughput (samples/s)	[MEASURE]	[MEASURE]	Samples / elapsed seconds
Model size (MB)	[MEASURE]	[MEASURE]	Serialized file size
Peak memory (MB)	[MEASURE]	[MEASURE]	Runtime memory profiler
Accuracy change	—	[MEASURE]	Edge accuracy minus full accuracy

50 ms acceptance decision

The model meets the latency requirement only if the measured p95 end-to-end latency is at or below 50 ms on the target device under the intended operating load. A mean below 50 ms is not enough if p95 exceeds the limit. Also check memory, thermal throttling and sustained throughput.

Example interpretation wording

The compressed model reduced serialized size by [MEASURE]% and changed macro-F1 by [MEASURE] percentage points. Its p95 latency was [MEASURE] ms, therefore it [met/did not meet] the 50 ms target. The main observed error increase was [CLASS/CONDITION]. Further work should focus on [QUANTISATION CALIBRATION / DATA BALANCING / ARCHITECTURE / PREPROCESSING].

Deployment safeguards

- Return 'uncertain' when confidence is low rather than forcing a label.
- Do not use emotion estimates as the sole basis for consequential decisions.
- Provide a clear notice and privacy controls for camera processing.
- Monitor drift and provide a fallback to a human operator.

PART B – TASK B4: RETRIEVAL-BASED CHATBOT

B4.1 Eight or more intents

Build a small retrieval-based operator assistant using TF-IDF vectors and cosine similarity. Each intent has example utterances and a response template. The assistant retrieves the closest intent only when its similarity is above a confidence threshold; otherwise it returns an out-of-scope fallback.

Intent	Example utterance	Response purpose
OPEN_ALERTS	Which alerts are open?	List active alerts from alert store.
ALERT_DETAILS	Show alert 104	Return timestamp, source and status.
WHY_FLAGGED	Why was this flagged?	Explain triggered rules/features.
SYSTEM_STATUS	Is the detector online?	Show health and last heartbeat.
MODEL_CONFIDENCE	What is the confidence?	Return score and threshold.
ACKNOWLEDGE_ALERT	Acknowledge alert 104	Record operator acknowledgement.
HELP	What can I ask?	List supported intents.
ESCALATE	Contact the supervisor	Escalate to authorised human.
CLOSE_ALERT	Close alert 104	Close only if user is authorised.

Core TF-IDF retrieval code

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

examples = [
  "which alerts are open", "show active alerts",
  "show alert details", "tell me about alert",
  "why was this flagged", "explain this detection",
  "is the system online", "check detector status",
  "what is the confidence", "show model score",
  "acknowledge this alert", "mark alert as seen",
  "help", "what can I ask",
  "contact supervisor", "escalate this case",
  "close this alert", "resolve alert"
]
labels = ["OPEN_ALERTS"]*2 + ["ALERT_DETAILS"]*2 + ["WHY_FLAGGED"]*2 + \
         ["SYSTEM_STATUS"]*2 + ["MODEL_CONFIDENCE"]*2 + \
         ["ACKNOWLEDGE_ALERT"]*2 + ["HELP"]*2 + ["ESCALATE"]*2 + ["CLOSE_ALERT"]*2
vectorizer = TfidfVectorizer(ngram_range=(1,2), lowercase=True)
X = vectorizer.fit_transform(examples)

def predict_intent(text, threshold=0.35):
    q = vectorizer.transform([text])
    sims = cosine_similarity(q, X).ravel()
    i = sims.argmax()
    if sims[i] < threshold:
        return "OUT_OF_SCOPE", float(sims[i])
    return labels[i], float(sims[i])
```

In production, protect state-changing intents such as acknowledge or close with authentication, authorisation, confirmation and audit logging. The retrieval example is a baseline, not a complete secure assistant.

TASK B4 – ASR ERRORS, THRESHOLD SWEEP AND DEPLOYMENT

B4.2 Simulate speech-recognition errors

Create corrupted versions of test utterances at word-error rates (WER) of 0%, 10%, 20% and 30%. WER = (substitutions + deletions + insertions) / number of words in the reference. Apply corruption after splitting the dataset so that no test utterance is used to build training examples.

WER	Intent accuracy	Out-of-scope recall	False-accept rate
0%	[MEASURE]	[MEASURE]	[MEASURE]
10%	[MEASURE]	[MEASURE]	[MEASURE]
20%	[MEASURE]	[MEASURE]	[MEASURE]
30%	[MEASURE]	[MEASURE]	[MEASURE]

False-accept rate is the fraction of genuinely out-of-scope utterances incorrectly assigned a supported intent. Sweep the similarity threshold, for example 0.10 to 0.90 in increments of 0.05, and plot supported-intent accuracy against false-accept rate. Choose a threshold using validation data and state the operational trade-off.

B4.3 Edge versus cloud decision

Factor	Edge deployment	Cloud deployment
Latency	Low network dependence; useful for immediate response.	Network round-trip may add delay.
Privacy	Raw images/audio can remain on device.	Data transmission requires encryption, access control and retention policy.
Compute	Limited memory, power and thermal capacity.	Can use larger models and elastic compute.
Availability	Can continue during network outage if local model is available.	Depends on connectivity and cloud service availability.
Cost	Device hardware and maintenance cost.	Usage, storage, bandwidth and service costs.
Updates	Model updates may be slower and device-specific.	Central updates are easier to manage.

Recommended hybrid decision: run a compact detection model and basic alert triage on the edge when the measured p95 latency meets 50 ms and privacy is important. Send only necessary metadata or authorised evidence to the cloud for long-term analytics, model retraining and complex review. Keep the operator interface available locally with a safe fallback when offline.

Reverse the decision if the edge model cannot meet accuracy or memory requirements, if thermal throttling causes p95 latency to exceed the limit, or if the device cannot be maintained securely. Move more inference to cloud only when connectivity, privacy controls and response-time requirements permit it.

Production controls

- Encrypt data in transit and at rest; minimise retention of face/audio data.
- Authenticate operators and enforce role-based permissions.
- Log model version, confidence, alert ID and human action.
- Monitor latency, false alarms, missed detections and data drift.
- Provide manual override, escalation and a documented rollback plan.

CONCLUSION, SUBMISSION CHECKLIST AND REFERENCES

Overall conclusion

A deployable intelligent system requires more than a model that produces an answer. The drone agent combines reactive safety with deliberative planning; the rescue swarm uses local decisions to tolerate communication failures; the expert-system shell separates reusable inference machinery from domain knowledge; and the RDD pipeline handles large sensor streams through lazy transformations, shuffles, caching and lineage.

For edge AI, compression can reduce memory and latency but may reduce accuracy or robustness. Therefore the full and compressed models must be evaluated on the same leakage-safe test set and measured on the actual target hardware. A retrieval-based chatbot adds an operator interface, but confidence thresholds, speech errors, security and human escalation must be tested before deployment.

Final submission checklist

- Part A: PEAS, environment classification and agent-type justification completed.
- Part A: percepts, internal state, actions, priorities, full pseudocode and six-step trace included.
- Part A: swarm diagnosis, local rule, neighbour information, grid trace and failure configuration included.
- Part A: three domain rule sets, generic shell, inference pseudocode, HOW/WHY and knowledge acquisition included.
- Part A: RDD transformations/actions, narrow/wide dependencies, caching and lineage explained.
- Part A: full-model estimate, compressed pipeline and trade-off described.
- Part B: source code, test inputs, expected and actual outputs attached.
- Part B: at least 30 seeds and required drone fleet/failure settings measured.
- Part B: 15 training and 15 hold-out expert cases documented; adversarial tests and regression results included.
- Part B: dataset source, leakage-safe split, metrics, confusion matrix and five real errors reported.
- Part B: PySpark sizes and partition counts benchmarked; caching and shuffle counts recorded.
- Part B: full and compressed model mean/p95 latency, throughput, size and accuracy compared.
- Part B: chatbot has at least eight intents, fallback threshold, intent metrics and WER stress tests.
- Part B: edge/cloud decision defended with measured evidence.
- All [MEASURE] fields replaced with actual results; code and environment details included.

References and tools

7. Russell, S. and Norvig, P., Artificial Intelligence: A Modern Approach, 4th edition, Pearson. (Agent architectures, search and intelligent systems.)
8. Spark Documentation, Apache Spark – RDD Programming Guide. https://spark.apache.org/docs/latest/rdd-programming-guide.html
9. scikit-learn Documentation – TfidfVectorizer and cosine_similarity. https://scikit-learn.org/stable/
10. TensorFlow Lite / LiteRT documentation – model optimisation and post-training quantisation. https://ai.google.dev/edge/litert
11. FER-2013 dataset: verify the original publication, dataset provenance, licence and current access terms before using it; cite the exact dataset version actually downloaded.
12. Python documentation. https://docs.python.org/3/

Academic integrity note: cite the exact dataset and software versions used. The design examples and code templates should be adapted, executed, understood and defended by the student. Do not present illustrative rows or placeholder metrics as experimental findings.